

Chapter 2 - Operators (Questions and Answers)

Pregunta número 1

1. Which of the following Java operators can be used with boolean variables? (Choose all that apply.)
- A. ==
 - B. +
 - C. --
 - D. !
 - E. %
 - F. ~
 - G. Cast with (boolean)

Answer 1: La primera respuesta correcta es el inciso A, ya que el operador de igualdad (==) es válido para comparar dos variables booleanas. La segunda respuesta correcta es el inciso D, ya que el operador lógico de negación está diseñado específicamente para variables booleanas; su función es invertir el estado lógico, por lo que !true se convierte en false y !false se convierte en true. Finalmente, la última respuesta correcta es el inciso G, ya que, aunque parezca redundante, el compilador permite aplicar el operador de conversión de tipo cast de un booleano hacia otro booleano explícitamente; esto en la práctica no es útil, pero sí es legal para el compilador.

Pregunta número 2

2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)
- ```
byte apples = 5;
short oranges = 10;
_____ bananas = apples + oranges;
```
- A. int
  - B. long
  - C. boolean
  - D. double
  - E. short
  - F. byte

**Answer 2:** Aquí tenemos una regla de oro del compilador de Java llamada Promoción Numérica (Numeric Promotion). La regla dicta que cuando aplicamos operadores aritméticos binarios (como la suma +) a tipos de datos enteros pequeños

(byte, short o char), Java promueve automáticamente ambos operandos a int antes de realizar la operación, sin importar si ninguno de los dos era int originalmente. Ahora bien, tenemos una variable de tipo byte y una variable de tipo short; cuando intentamos sumarlas, el compilador convierte a ambas variables en int, por lo tanto, el resultado de esa suma es de tipo int. Con esto podemos decir que la primera respuesta correcta es el inciso A, ya que el resultado de la operación será un int y esto encaja perfectamente en una variable de tipo int. Las otras dos respuestas correctas son los incisos B y D, esto debido a que Java permite la conversión implícita (automática) de un tipo de dato más pequeño a uno más grande. Un int cabe sin problemas de pérdida de precisión dentro de un long (64 bits enteros) o un double (64 bits de punto flotante). El compilador acepta esto de manera silenciosa.

### Pregunta número 3

3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

```
3: long ear = 10;
```

```
4: int hearing = 2 * ear;
```

- A. No change; it compiles as is.
- B. Cast ear on line 4 to int.
- C. Change the data type of ear on line 3 to short.
- D. Cast 2 \* ear on line 4 to int.
- E. Change the data type of hearing on line 4 to short.
- F. Change the data type of hearing on line 4 to long.

**Answer 3:** Aquí vuelven a entrar las reglas de promoción numérica, pero esta vez con un tipo de dato más grande. La regla nos dice que si uno de los operandos en una operación aritmética es de tipo long, Java promueve automáticamente el otro operando a long, y el resultado final de la operación también será un long. En el código tenemos una variable de tipo int y otra de tipo long; al multiplicarlas, el compilador va a convertir a la variable de tipo int en tipo long, y el resultado de la multiplicación será un long. El problema ocurre en la propia línea 4 porque estamos intentando guardar un long en una variable de tipo int, lo cual el compilador prohíbe para evitar pérdida de datos. La primera respuesta correcta es el inciso B, ya que si casteamos la variable `ear` de tipo `long` a tipo `int`, tendríamos dos valores de tipo `int`; al multiplicarlos, seguiríamos teniendo un valor de tipo `int` que se podría almacenar perfectamente en la variable `hearing`, que es de tipo `int`. La siguiente respuesta correcta es el inciso C, ya que si cambiamos a era a tipo short, la operación sería int \* short. Debemos recordar que Java promueve los tipos pequeños a int en las matemáticas. Así que la operación da como resultado un int, solucionando el problema. La siguiente respuesta correcta es la letra D, ya que si

hacemos un casteo a int de todo el resultado de  $2 * ear$ ; dejamos que la multiplicación suceda como long, pero justo después forzamos con el cast explícito a que el resultado final sea un int justo antes de guardarlo. Al compilador le parece perfecto porque nosotros asumimos la responsabilidad de la posible pérdida de precisión. Finalmente, la última respuesta correcta es el inciso F, ya que aquí simplemente cambiamos el tipo de la variable hearing a long; esta es la solución más natural. Si la multiplicación devuelve un long, simplemente cambiamos el tipo de la variable destino a long para que coincidan.

#### Pregunta número 4

4. What is the output of the following code snippet?

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+", "+teeth+", "+wolf);
```

- A. true, 20, true
- B. true, 20, false
- C. false, 10, true
- D. false, 20, false
- E. The code will not compile because of line 5.
- F. None of the above.

**Answer 4:** La clave de esta pregunta radica en la línea 5; lo primero que tenemos que hacer es dividir en partes esa línea para entender lo que se está realizando. Empezamos con `teeth != 10`; recordemos que la variable `teeth` vale 20, por lo que la comparación de 20 es diferente a 10, por lo que esta parte se evalúa como `true`. Del otro lado tenemos la asignación de `false` a la variable `wolf`; después hay que ver la parte del operador `^`, que es un XOR y OR exclusivo; entonces podemos concentrar toda esa línea de código como `canine = true ^ false`. La regla del XOR para booleanos dicta que se devolverá `true` únicamente si los operandos son diferentes; en este caso puntual, como son diferentes, la evaluación final será `true`. Dicho valor `true` se almacenará en la variable `canine`. Entonces, al llegar a la línea 6, el estado de las variables es `canine = true`, `teeth = 20`, ya que nunca se modificó, solo se comparó, y `wolf = false`, ya que esta variable se sobrescribe en la línea 5. Por lo tanto, la respuesta correcta es el inciso B.

#### Pregunta número 5

5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)
- A. +, \*, %, --
  - B. ++, (int), \*
  - C. =, ==, !
  - D. (short), =, !, \*
  - E. \*, /, %, +, ==
  - F. !, |, &
  - G. ^, +, =, +=

**Answer 5:** Aquí nos adentramos en la tabla de precedencia de operadores de Java. La precedencia determina qué operaciones se evalúan primero en una expresión. En esta pregunta puntual nos están indicando que debemos identificar las opciones donde los operadores están ordenados en precedencia creciente o igual (increasing or the same order). Esto significa ir desde los operadores que se evalúan al final (menor prioridad) hacia los que se evalúan primero (mayor prioridad). La primera respuesta correcta es el inciso A, ya que empieza con la suma binaria (+) que tiene precedencia baja, sigue con multiplicación y módulo (\* y %) que tienen una precedencia mayor que la suma, y ambos están en el mismo nivel de precedencia entre sí. De ahí nos vamos al decremento unario (--); recordemos que los operadores unarios tienen de las precedencias más altas en Java. Entonces tenemos un orden que va de baja, pasa por media y termina en alta. La otra respuesta correcta es el inciso C, ya que inicia con la asignación (=), que tiene la precedencia más baja de todas, ya que la asignación siempre es lo último que ocurre. De ahí sigue la igualdad (==), que tiene precedencia media, ya que se evalúa antes de asignar, pero después de hacer las matemáticas. Termina con la negación lógica/operador unario (!) que tiene de las precedencias más altas. Con esto comprobamos que el orden va de muy baja, media y termina en alta, lo que significa que la precedencia va en aumento. Entonces tenemos que las respuestas correctas son los incisos A y C.

### Pregunta número 6

6. What is the output of the following program?

```
1: public class CandyCounter {
2: static long addCandy(double fruit, float vegetables) {
3: return (int)fruit+vegetables;
4: }
5:
6: public static void main(String[] args) {
7: System.out.print(addCandy(1.4, 2.4f) + ", ");
8: System.out.print(addCandy(1.9, (float)4) + ", ");
9: System.out.print(addCandy((long)(int)(short)2, (float)4)); } }
```

- A. 4, 6, 6.0
- B. 3, 5, 6
- C. 3, 6, 6
- D. 4, 5, 6
- E. The code does not compile because of line 9.
- F. None of the above.

**Answer 6:** Aquí estamos viendo tres conceptos fundamentales; estos son la precedencia de operadores, la promoción numérica y la compatibilidad de tipos de retorno. Este código genera un error de compilación en la línea 3; hay que desglosar el porqué. Lo primero tiene que ver con la precedencia: el operador de conversión o cast (int) tiene mayor prioridad que el operador de suma (+). Por lo tanto, Java evalúa la expresión como ((int) fruit) + vegetables. Lo siguiente a tomar en cuenta es que la variable fruit es de tipo double; al aplicarle el cast (int), se convierte en un int (truncando sus decimales). De ahí entramos a la parte de la promoción numérica, ya que ahora tenemos una suma entre un int y la variable vegetables, que es de tipo float. Recordemos que Java promueve ambos al tipo de dato más grande, que en este caso es float. Por lo tanto, el resultado final de la suma es float. Y finalmente viene el error de compatibilidad; esto sucede, ya que el método addCandy declara explícitamente en la línea 2 que va a retornar un long. Entonces, al intentar devolver un float en un método que espera un long, causará un error inmediato en el compilador. Convertir de un tipo de dato con punto flotante (float) a uno entero (long) requiere obligatoriamente un cast explícito. Con lo anterior presente, la respuesta correcta es el inciso F.

### Pregunta número 7

7. What is the output of the following code snippet?

```
int ph = 7, vis = 2;
boolean clear = vis > 1 & (vis < 9 || ph < 2);
boolean safe = (vis > 2) && (ph++ > 1);
boolean tasty = 7 <= --ph;
System.out.println(clear + "-" + safe + "-" + tasty);
```

- A. true-true-true
- B. true-true-false
- C. true-false-true
- D. true-false-false
- E. false-true-true
- F. false-true-false
- G. false-false-true
- H. false-false-false

**Answer 7:** En esta pregunta se nos está evaluando la comprensión del cortocircuito lógico (short-circuiting) combinado con los operadores de incremento y decremento. Para estas preguntas nos conviene ir revisando línea por línea para ir resolviendo cada parte. Al evaluar la variable `clear`, primero resolvemos los paréntesis; esa parte nos devuelve `true`. Como tenemos un operador OR, al ser el lado izquierdo `true`, el paréntesis completo es `true` (no importa si `ph < 2` es falso). Ahora, yendo a la parte “exterior”, tenemos que es `true`, ya teniendo que `true` y `true`, pues tenemos como resultado `true`. Entonces, el estado actual hasta esta parte es que `clear = true`, `ph = 7` y `vis = 2`. Pasando a evaluar `tasty`, primero descubrimos que `ph` sigue valiendo 7 gracias a la línea anterior. Tenemos el operador `--ph`, que es un pre-decremento. Esto significa que primero resta 1 a la variable y luego realiza la comparación; entonces, `ph` pasa de 7 a 6. Se evalúa la expresión `7 <= 6`, lo cual es `false`; hasta esta parte, el estado actual es que `tasty = false` y `ph = 6`. Con esto podemos saber que, al imprimir la concatenación final, el resultado nos dará `true-false-false`; por lo tanto, la respuesta correcta es el inciso D.

### Pregunta número 8

8. What is the output of the following code snippet?

```
4: int pig = (short)4;
5: pig = pig++;
6: long goat = (int)2;
7: goat -= 1.0;
8: System.out.print(pig + " - " + goat);
```

- A. 4 - 1
- B. 4 - 2
- C. 5 - 1
- D. 5 - 2
- E. The code does not compile due to line 7.
- F. None of the above.

**Answer 8:** Las claves de esta pregunta son el comportamiento del postincremento cuando se reasigna a la misma variable y el casteo automático de los operadores de asignación compuesta. En la línea 4 se asigna el número 4 a la variable pig, se hace el cast explícito a short; es legal, pero termina siendo inútil, esto debido a que, como pig es int, el compilador lo promueve (widening) automáticamente de regreso a int. Hasta aquí, pig = 4. La línea 5 funciona en dos pasos; el primero es que devuelve el valor original de la variable (que es 4) para que se use en la expresión. El segundo paso es que se incrementa la variable en memoria (pasándola a 5); sin embargo, tenemos un operador de asignación. El valor que devolvió la expresión (4) se le asigna a pig, sobrescribiendo el 5 que acababa de calcularse en memoria. Lo que nos da como resultado final es que pig se queda con el valor de 4. En la línea 6, al igual que en la línea 4, asignamos el valor 2. El cast (int) es redundante, y el 2 se promueve a long sin problemas para caber en goat. Hasta aquí, goat = 2 L. En la línea 7, los operadores de asignación compuesta aplican un cast explícito automático hacia el tipo de dato de la variable receptora. Para el compilador, la línea se traduce exactamente a goat = (long) (goat - 1.0); lo importante de aquí es tener presente que, al forzar el cast, no hay error de compilación. El resultado nos da 1. Finalmente, en la línea 8, al imprimir pig + " - " + goat, el resultado final es 4 - 1. Con estos datos podemos decir que la respuesta correcta es el inciso A.

### Pregunta número 9

9. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a!=c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. The code does not compile.

**Answer 9:** Tenemos nuestro estado inicial  $a = 2$ ,  $b = 4$  y  $c = 2$ ; en la primera impresión, la condición  $2 > 2$  es falsa. Se ejecuta  $b++$ . Por ser post-incremento, primero imprime el valor original 4 y luego  $b$  sube a 5. En la segunda impresión tenemos la condición  $2 \neq 2$  es falsa. Se ejecuta  $b++$ . El postincremento devuelve el valor actual (5) y sube "b" a 6 en memoria. La trampa está en la asignación explícita  $b = \dots$ . Toma ese 5 devuelto y lo guarda en  $b$ , sobrescribiendo el 6. Se imprime 5. En la tercera impresión, la condición principal  $2 > 5$  es falsa. El código ignora por completo el ternario anidado y salta directo al valor de la rama falsa principal. Se imprime 1. Con lo anterior dicho, tenemos que las respuestas correctas son los incisos B, C y F.

### Pregunta número 10



10. What are the unique outputs of the following code snippet? (Choose all that apply.)

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

- A. 1
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. The code does not compile.

**Answer 10:** En este código tenemos la falla en la segunda línea. Primero debemos recordar la promoción numérica, que nos dice que cualquier operación matemática que involucre tipos de datos pequeños (byte, short, char) hace que Java promueva automáticamente los operandos a int antes de realizar la operación. Por lo tanto, aunque forcemos las variables a ser byte, al multiplicarlas el resultado se convierte en un int. Intentar guardar ese int devuelto dentro de la variable zebra (que es short) sin un cast explícito genera un error inmediato en el compilador por posible pérdida de precisión. Con esta información podemos afirmar sin miedo que la respuesta correcta es el inciso G.

### Pregunta número 11

**11.** What is the output of the following code?

```
11: int sample1 = (2 * 4) % 3;
12: int sample2 = 3 * 2 % 3;
13: int sample3 = 5 * (1 % 2);
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

- A. 0, 0, 5
- B. 1, 2, 10
- C. 2, 1, 5
- D. 2, 0, 5
- E. 3, 1, 10
- F. 3, 2, 6
- G. The code does not compile.

**Answer 11:** Para resolver esta pregunta debemos tener en cuenta las reglas de precedencia de operadores, específicamente enfocándonos en los paréntesis (), la multiplicación \* y el operador de módulo/residuo %. Debemos recordar que \* y % tienen exactamente el mismo nivel de precedencia, por lo que cuando se encuentran juntos sin paréntesis, se evalúan de izquierda a derecha. Yendo línea por línea, tenemos primero que en la línea 11 los paréntesis tienen la prioridad más alta. Primero evaluamos  $(2 * 4)$ , lo que nos da 8; ahora la expresión es  $8 \% 3$ . Al dividir 8 entre 3, el cociente es 2 y el residuo es 2; el resultado es  $\text{sample1} = 2$ . De ahí, en la línea 12, debemos primero tener en cuenta que aquí no hay paréntesis. Como \* y % tienen la misma prioridad, leemos de izquierda a derecha. Primero se hace la multiplicación.  $3 * 2$ , que nos da 6. Ahora aplicamos el módulo al resultado de  $6 \% 3$ . Al dividir 6 entre 3, la división es exacta, por lo que el residuo es 0. El resultado de  $\text{sample2} = 0$ . De ahí, en la línea 13, tenemos que resolver el paréntesis  $(1 \% 2)$ . Al dividir 1 entre 2, el cociente es 0 y nos sobra 1 (siempre que hacemos el módulo de un número menor entre uno mayor, el resultado es el número menor). Ahora continuamos con la multiplicación de  $5 * 1$ , que nos da 5; el resultado es  $\text{sample3} = 5$ . Al imprimir la concatenación, el resultado es 2, 0, 5. Entonces la respuesta correcta es el inciso D.

### Pregunta número 12

12. The \_\_\_\_\_ operator increases a value and returns the original value, while the \_\_\_\_\_ operator decreases a value and returns the new value.
- A. post-increment, post-increment
  - B. pre-decrement, post-decrement
  - C. post-increment, post-decrement
  - D. post-increment, pre-decrement
  - E. pre-increment, pre-decrement
  - F. pre-increment, post-decrement

**Answer 12:** En esta pregunta se evalúan directamente las definiciones teóricas de los operadores unarios de incremento y decremento. La respuesta correcta es el inciso D, ya que en esta se nos da la definición exacta del post-incremento (variable ++). Al usarlo en una expresión, primero se entrega el valor actual de la variable a la expresión, y luego la variable sube en 1 en la memoria. También se complementa con la definición exacta del pre-decremento (--variable). Al usarlo, primero se le resta 1 a la variable en memoria, y ese nuevo valor ya modificado es el que se devuelve a la expresión.

### Pregunta número 13

13. What is the output of the following code snippet?

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
 + "-" +stayingHome);
```

- A. true-false-false
- B. false-true-false
- C. true-true-true
- D. false-true-true
- E. false-false-false
- F. true-true-false
- G. None of the above

**Answer 13:** En este ejercicio vamos a repasar la precedencia de los operadores lógicos a nivel de bits cuando se usan con valores booleanos. Aquí es primordial evaluar cada variable paso a paso con sus valores iniciales: sunny = true, raining = false, sunday = true. Evaluando goingToTheStore al reemplazar los valores de la

expresion nos queda `true & false ^ true`. Contemplando la precedencia, el operador AND lógico sin cortocircuito (`&`) tiene mayor prioridad que el operador XOR (`^`); por lo tanto, se evalúa primero. Entonces, el paso 1 es que al evaluar `true %=& false`, da como resultado `false`; en el paso 2 nos toca evaluar el XOR con el resultado anterior: `false ^ true`. Como vimos en preguntas pasadas, el XOR devuelve `true` si los valores son diferentes. Entonces el resultado es `goingToTheScore = true`. Ahora continuamos con `goingToTheZoo`; al reemplazar los valores de la expresión, nos da `true && !false`. La precedencia nos dice que el operador unario de negación (`!`) tiene prioridad alta, por lo que se evalúa antes que el AND lógico (`&&`); entonces debemos comenzar con que `!false` se convierte en `true`. El siguiente paso que tenemos es que `true && true` da como resultado `true`. Esto nos da como resultado que `goingToTheZoo = true`. Siguiendo con la evaluación, le toca el turno a `stayingHome`; reemplazando con los resultados que acabamos de calcular tenemos `!(true && true)`. El primer paso es el paréntesis; evaluamos lo que está dentro de los paréntesis: tenemos que `true && true` es `true`. El siguiente paso es aplicar la negación al resultado del paréntesis; por lo tanto, `!true` es `false`. Con esto en mente, podemos decir que el resultado es `stayingHome = false`. Entonces, al imprimir la concatenación `goingToTheStore + "-" + goingToTheZoo + "-" + stayingHome`, nos da que el resultado final es `true-true-false`. Con todo lo anterior podemos afirmar que la respuesta correcta es el inciso F.

#### Pregunta número 14

14. Which of the following statements are correct? (Choose all that apply.)
- A. The return value of an assignment operation expression can be `void`.
  - B. The inequality operator (`!=`) can be used to compare objects.
  - C. The equality operator (`==`) can be used to compare a `boolean` value with a numeric value.
  - D. During runtime, the `&` and `|` operators may cause only the left side of the expression to be evaluated.
  - E. The return value of an assignment operation expression is the value of the newly assigned variable.
  - F. In Java, `0` and `false` may be used interchangeably.
  - G. The logical complement operator (`!`) cannot be used to flip numeric values.

**Answer 14:** La primera respuesta correcta es el inciso B, ya que en Java podemos usar `==` y `!=` para comparar referencias de objetos. Si usamos `objeto1 != objeto2`, el compilador evalúa si ambas referencias apuntan a espacios de memoria diferentes. La segunda respuesta correcta es el inciso E, ya que una asignación en Java no solo guarda un valor, sino que también devuelve ese mismo valor. Por eso es válido hacer asignaciones en cadena como `a = b = c = 5`, porque `c = 5` devuelve 5, que luego se asigna a `b`, y así sucesivamente. Finalmente, la tercera respuesta correcta

es el inciso G, ya que en Java, el operador de negación lógica (!) es estrictamente exclusivo para variables boolean. Si queremos invertir un valor numérico (a nivel de bits), debemos usar el operador de complemento a nivel de bits (~).

### Pregunta número 15

15. Which operators take three operands or values? (Choose all that apply.)

- A. =
- B. &&
- C. \*=
- D. ? :
- E. &
- F. ++
- G. /

**Answer 15:** En esta pregunta estamos indagando sobre los operadores que requieren exactamente tres operandos (valores o expresiones) para poder funcionar. A estos se les conoce formalmente como operadores ternarios. Ahora bien, la respuesta correcta es el inciso D ( ? : ) , ya que en Java, este es el único operador ternario que existe (también conocido como operador condicional). Su estructura obligatoriamente requiere tres partes: Una condición booleana a evaluar, el valor/expresión que se devolverá si es true y el valor/expresión que se devolverá si es false.

### Pregunta número 16

16. How many lines of the following code contain compiler errors?

```
int note = 1 * 2 + (long)3;
short melody = (byte)(double)(note *= 2);
double song = melody;
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

**Answer 16:** El error se encuentra en la primera línea de código; esta línea falla por las reglas de promoción numérica. La multiplicación `1 * 2` se evalúa como un `int`

(valor 2). Sin embargo, al sumarlo con (long)3, Java promueve automáticamente el resultado de toda la expresión del lado derecho a long (con un valor de 5L). Intentar guardar un long dentro de una variable int (note) sin un cast explícito genera un error inmediato de compilación por posible pérdida de datos. Es importante mencionar que las otras líneas de código sí logran engañar al compilador, por ejemplo, la línea 2, aunque visualmente es un caso, el punto clave es que al asignar un resultado byte a una variable short (melody), es totalmente válido porque el compilador hace una ampliación de tipo (widening) automática, ya que un byte cabe perfectamente en un short. En la siguiente línea, la clave es sencilla: pasar un short a un double es una ampliación de tipo automática, por lo que compila sin problemas. Finalmente, la última línea tiene varias trampas, pero ninguna genera un error. Primero, el 1\_000f es un literal válido, ya que Java permite guiones bajos para mejorar la legibilidad de los números. El segundo, el operador ternario evalúa si devuelve song \* 2L (un double multiplicado por un long da como resultado un double) o si devuelve song (que ya es double). Por lo tanto, el resultado total del operador ternario es un double. El tercero, antes de asignarlo a la variable symphony (que es float), hay un cast explícito (float) envolviendo todo el ternario. Al asumir la responsabilidad de convertir el double a float, el compilador lo hace sin problemas. Con esto en mente, podemos decir que la respuesta correcta es el inciso B, ya que solo una línea va a generar un error de compilación.

### Pregunta número 17

17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

- A. ticketsSold is 8.
- B. ticketsTaken is 2.
- C. ticketsSold is 6.
- D. ticketsTaken is 6.
- E. ticketsSold is 7.
- F. ticketsTaken is 4.
- G. The code does not compile.

**Answer 17:** Vamos a seguir el flujo del código para entender lo que se está haciendo. El estado inicial de las variables ticketsTaken = 1 y ticketSold = 3. En la línea 3, primero evaluamos la expresión de la derecha: 1 + ticketsTaken++. Como es un post-incremento, la variable ticketsTaken devuelve su valor original para la suma.

(1) y luego sube a 2 en la memoria. La suma da como resultado 2. Ahora aplicamos la reasignación compuesta: `ticketsSold += 2` significa que `ticketsSold = 3 + 2`, lo que da 5. Hasta aquí, el estado actual es `ticketsSold = 5` y `ticketsTaken = 2`. En la línea 4 tenemos `ticketsTaken *= 2`; es simplemente como colocar `ticketsTaken = 2 * 2`. El estado actual: `ticketsSold = 5` y `ticketsTaken = 4`. De ahí, en la línea 5, `ticketsSold += (long)1`, estamos evaluando la regla del cast automático. Sumar un long a un int matemáticamente da como resultado un long. Sin embargo, como estamos usando el operador `+=`, Java hace un cast automático de regreso al tipo original de la variable receptora (int). Por lo tanto, la operación `5 + 1L` da `6L`, se hace el cast a int sin problemas y `ticketsSold` se convierte en 6. Entonces, al final de la ejecución, `ticketsSold` vale 6 y `ticketsTaken` vale 4. Ya habiendo seguido el flujo del programa, podemos afirmar que la respuesta correcta es el inciso C.

### Pregunta número 18

18. Which of the following can be used to change the order of operation in an expression? (Choose all that apply.)
- A. [ ]
  - B. < >
  - C. ( )
  - D. \ /
  - E. { }
  - F. " "

**Answer 18:** En Java, los paréntesis redondos ( ) son el único mecanismo que el compilador reconoce para alterar explícitamente el orden natural de evaluación (precedencia) en una expresión. Al encerrar una subexpresión entre paréntesis, le indicamos al compilador que resuelva esa parte primero, saltándose la tabla de precedencia habitual. Todos los demás símbolos tienen propósitos completamente distintos y el compilador arrojaría error si intentamos usarlos para agrupar operaciones matemáticas o lógicas. Los corchetes cuadrados [] se utilizan exclusivamente para declarar, instanciar o acceder a los elementos de un arreglo. Los símbolos de menor y mayor <> se usan como operadores relacionales numéricos o para definir tipos genéricos (generics). La barra invertida \ es un carácter de escape para cadenas de texto y la barra normal / es el operador aritmético de división. Las llaves o corchetes rizados se usan para definir bloques de alcance (scope) en el código (como el cuerpo de una clase, un método, un if o un for) o para inicializar arreglos, pero no agrupan expresiones aritméticas. Por último, las comillas dobles se usan estrictamente para declarar literales de texto (objetos de tipo String). Por lo tanto, la respuesta correcta es el inciso C.

### Pregunta número 19

19. What is the result of executing the following code snippet? (Choose all that apply.)

```
3: int start = 7;
4: int end = 4;
5: end += ++start;
6: start = (byte)(Byte.MAX_VALUE + 1);
```

- A. start is 0.
- B. start is -128.
- C. start is 127.
- D. end is 8.
- E. end is 11.
- F. end is 12.
- G. The code does not compile.
- H. The code compiles but throws an exception at runtime.

**Answer 19:** Aquí los conceptos clave son el preincremento y el desbordamiento numérico (numeric overflow). Empezando en la línea 3 y 4, tenemos `start = 7` y `end = 4`; de ahí, en la línea 5, tenemos un preincremento con `++start`. Esto significa que primero incrementamos la variable `start` en la memoria (pasa de 7 a 8) y ese nuevo valor (8) es el que se devuelve a la expresión. Resolviendo la asignación compuesta, tenemos que `end += 8` equivale a `end = 4 + 8`. Por lo tanto, `end` termina valiendo 12 y vuelve al inciso F como una respuesta correcta. Siguiendo con la línea 6, tenemos la trampa del desbordamiento; sabemos que en `Byte.MAX_VALUE` en Java es 127. Al sumarle 1, la operación matemática  $127 + 1$  da como resultado el int 128. El problema es que un tipo `byte` solo puede almacenar valores desde -128 hasta 127. No tiene espacio para el 128. Al hacer el casto explícito (`byte`), le estás obligando al compilador a meter ese 128 en un `byte`. En Java, cuando un número excede el límite máximo de su tipo de dato, sufre un desbordamiento (overflow) y da la vuelta hacia el extremo opuesto. El número que le sigue al máximo positivo (127) da la vuelta completa y se convierte en el mínimo negativo (-128). Por lo tanto, `start` termina valiendo -128. Con estos datos podemos afirmar que la respuesta correcta es el inciso B.

### Pregunta número 20



20. Which of the following statements about unary operators are true? (Choose all that apply.)
- A. Unary operators are always executed before any surrounding numeric binary or ternary operators.
  - B. The `~` operator can be used to flip a boolean value.
  - C. The pre-increment operator (`++`) returns the value of the variable before the increment is applied.
  - D. The post-decrement operator (`--`) returns the value of the variable before the decrement is applied.
  - E. The `!` operator cannot be used on numeric values.
  - F. None of the above

**Answer 20:** Aquí estamos evaluando las definiciones y reglas de precedencia de los operadores unarios. La primera respuesta correcta es el inciso A, ya que los operadores unarios siempre se ejecutan antes de cualquier otro operador binario o ternario circundante. En la tabla de precedencia de Java, los operadores unarios (`++`, `--`, `!`, `~`, `+`, `-`) tienen el nivel de prioridad más alto de todos los operadores funcionales (solo los paréntesis de agrupación están por encima). La siguiente respuesta correcta es el inciso D, ya que el operador de postdecremento retorna el valor de la variable antes de que se aplique el decremento. En casos anteriores vimos que en `x--`, la expresión recibe el valor original de `x`, y posteriormente `x` se reduce en 1 en la memoria. La última respuesta correcta es el inciso E, ya que el operador `!` no se puede usar en valores numéricos; solo lo podemos usar en valores booleanos. La negación lógica es estricta y exclusivamente para tipos de datos boolean. Intentar usarlo en un `int` o `double` genera un error de compilación.

### Pregunta número 21

21. What is the result of executing the following code snippet?

```
int myFavoriteNumber = 8;
int bird = ~myFavoriteNumber;
int plane = -myFavoriteNumber;
var superman = bird == plane ? 5 : 10;
System.out.println(bird + "," + plane + "," + --superman);
```

- A. -7,-8,9
- B. -7,-8,10
- C. -8,-8,4
- D. -8,-8,5
- E. -9,-8,9
- F. -9,-8,10
- G. None of the above

**Answer 21:** La clave está en la diferencia del operador de complemento a nivel de bits (~) y el operador de negación matemática (-). La variable `int myFavoriteNumber` inicializa con un valor de 8. De ahí en `int bird = ~myFavoriteNumber`; aquí aplicamos el operador unario de complemento a nivel de bits (~). Este operador invierte todos los ceros y unos del número en su representación binaria. Debemos recordar que no necesitamos hacer la conversión a binario. La fórmula matemática rápida para saber el resultado de `~x` en Java es `-x - 1`. Aplicando la fórmula `-8 - 1 = -9`. El estado actual es `bird = -9`. De ahí, yendo a `int plane = -myFavoriteNumber`, tenemos que este es el operador unario negativo de toda la vida. Simplemente invierte el signo matemático del número. Nuestro estado actual es `plane = -8`. Entonces `var superman = bird == plane ? 5 : 10`; Ahora toca evaluar la condición del operador ternario `-9 == -8` es `false`. Como es falso, ignoramos el 5 y le asignamos el 10 a la variable. Entonces nuestro estado actual es `superman = 10`. Al llegar al momento de imprimir, aplicamos un pre-decremento (`--superman`). Esto significa que primero se le resta 1 a la variable en memoria (pasa de 10 a 9) y luego se devuelve ese nuevo valor a la expresión de impresión. Por lo tanto, la concatenación final es `-9 + , + -8 + , + 9`, lo que nos da como resultado final `-9, -8, 9`. Con lo anterior dicho, podemos decir que la respuesta correcta es el inciso E.